KL UNIVERSITY

Department of Computer Science and Engineering

PROJECT REPORT

AZURE MANAGED IDENTITY ADOPTION ACROSS AN APPLICATION ESTATE

Replacing Service Principal Secrets with Secretless Azure Authentication

Project Code	24CC3046-P030
Team	T234 (P-030)
Department	Computer Science and Engineering
Institution	KL University
Document Version	1.0
Date	September 2026

SUBMITTED BY

TEAM MEMBER	ROLL NUMBER
CHILAKALA GNANA SAI SIDDHARDHA REDDY	2400030896
BIRADAKOTA VIVEK SIVA DHANUSH	2400030886
KATRU SASIDHAR	2400031135
AKURATHI SIVA RAMA NAGESWARA RAO	2400031115

Academic Project Documentation

TABLE OF CONTENTS

1. Abstract

2. Introduction

3. Problem Statement

4. Project Aim, Objectives and Scope

5. Background and Technology Overview

6. Proposed System Architecture

7. Migration Methodology

8. Tools and Technologies

9. Security, Risk and Governance

10. Bottlenecks and Limitations

11. Validation and Test Plan

12. Expected Outcomes and Benefits

13. Project Deliverables and Future Enhancements

14. Conclusion

15. References

Appendix A — Implementation Evidence Checklist

Appendix B — Terms and Abbreviations

Document note: This report describes the proposed adoption approach. Implementation status and measured outcomes should be updated with verified project evidence before final institutional submission.

1. ABSTRACT

Cloud-hosted applications routinely authenticate to platform services such as Azure Key Vault, Azure Storage and Azure SQL Database. A common legacy approach uses Microsoft Entra application registrations and service principals authenticated with client secrets or certificates. Although workable, these credentials must be created, stored, distributed, monitored, rotated and revoked. This creates operational effort and exposes applications to risks associated with credential leakage, expiry and configuration errors.

This project examines the adoption of Azure Managed Identity as a secretless authentication approach for supported Azure workloads. A managed identity is represented in Microsoft Entra ID and is provisioned for an Azure resource. The workload obtains a short-lived access token from the platform-provided identity endpoint and presents that token to a target service that supports Microsoft Entra authentication. The application therefore does not need to store a long-lived client secret or certificate for that authentication path.

The report presents the problem, objectives, technical background, proposed architecture, migration methodology, security controls, validation plan, constraints and expected benefits. It distinguishes the proposed adoption design from completed implementation evidence: no estate inventory, deployment logs, test results or measured savings were included in the source material. These items are therefore identified as activities and evidence to be completed during implementation rather than claimed as achieved results.

2. INTRODUCTION

Cloud applications are rarely isolated. A web application may retrieve credentials from Key Vault, a background worker may write files to Blob Storage, and an API may query Azure SQL Database. Each interaction requires an identity and an authorization decision. Authentication establishes which workload is making the request; authorization determines what that workload is allowed to do.

In a conventional service-principal model, an application is represented in Microsoft Entra ID and uses a client secret or certificate to prove its identity when requesting an access token. The secret is a durable credential. It must be protected in configuration or a secret store, made available to the workload, rotated before expiry and revoked when no longer trusted. A large application estate may contain many such credentials across environments and teams.

Azure Managed Identity reduces this credential-management burden for supported Azure-hosted workloads. The platform manages the identity and provides a local endpoint through which the workload requests tokens. The identity still requires carefully scoped permissions on each target service; managed identity removes the need to manage an application-held secret, but it does not remove the need for access governance.

This project, undertaken by Team T234 (P-030), documents a structured approach to replacing service-principal secret-based authentication with managed identity wherever the workload and target service support it.

3. PROBLEM STATEMENT

The current secret-based authentication pattern introduces recurring operational and security challenges across an application estate:

- Credential sprawl: separate client secrets or certificates may exist for multiple applications, environments and downstream services.
- Secret exposure: credentials stored in configuration files, environment variables, deployment pipelines or secret stores may be disclosed through human error, excessive access or compromised systems.
- Expiry-related outages: an expired, rotated incorrectly or misconfigured credential can interrupt an application's access to a dependent service.
- Rotation overhead: teams must coordinate creation, deployment, validation and retirement of credentials. The effort increases as the number of workloads grows.
- Audit complexity: it can be difficult to identify credential owners, consumers, expiry dates, permissions and rotation status consistently.

The central problem is therefore the dependence on long-lived application credentials for Azure-to-Azure authentication, even when the workload platform can provide a managed identity.

4. PROJECT AIM, OBJECTIVES AND SCOPE

4.1 Project Aim

To define and support the adoption of Azure Managed Identity as the preferred secretless authentication mechanism for Azure-hosted applications accessing compatible Azure services, reducing reliance on manually managed service-principal secrets and certificates.

4.2 Objectives

– Inventory applications and service-principal credentials used for Azure service-to-service authentication.

– Classify workloads according to managed-identity support, target-service compatibility, environment and ownership.

– Select system-assigned or user-assigned identities according to workload lifecycle and sharing requirements.

– Grant only the permissions required by each workload, preferably through Azure role-based access control (RBAC) or the target service's supported authorization model.

– Update application configuration and code to acquire tokens through a supported Azure identity library or platform endpoint.

– Validate authentication, authorization, logging, failure handling and rollback before retiring existing credentials.

– Document exceptions, including legacy integrations and cross-tenant access requirements.

4.3 Scope

In scope: Azure-hosted workloads; managed identity enablement; identity selection; access assignment to compatible services; application configuration changes; testing; monitoring; and a migration inventory.

Out of scope: redesigning unsupported third-party systems, replacing every non-Azure credential in the organisation, and claiming production-wide completion without an approved estate inventory and deployment evidence.

4.4 Use Cases

- UC-01 — Retire eligible service-principal secrets: identify credentials used only for supported Azure resource access, migrate the workload, validate it and revoke the obsolete credential.
- UC-02 — Enable secretless Azure access: configure a workload to use a managed identity when accessing services such as Key Vault, Blob Storage and Azure SQL Database, subject to service and SDK support.

5. BACKGROUND AND TECHNOLOGY OVERVIEW

5.1 Microsoft Entra ID and Service Principals

Microsoft Entra ID (formerly Azure Active Directory) is Microsoft's cloud identity and access-management service. An application registration describes an application, while its service principal is the application identity in a particular tenant. A client secret or certificate can be used by an application to authenticate as that service principal and request OAuth 2.0 access tokens.

Service principals remain valid for many application scenarios, including non-Azure hosting and integrations that cannot use managed identity. The project does not treat service principals as inherently insecure; it targets avoidable long-lived secrets in workloads that can use a platform-managed identity.

5.2 Azure Managed Identity

A managed identity is a special type of service principal whose credentials are managed by Azure. The workload requests a token from an identity endpoint made available by its hosting environment. The resulting token is short-lived and intended for a specific resource audience. The target service validates the token and applies its authorization rules.

5.3 Types of Managed Identity

Attribute	System-assigned	User-assigned
Provisioning	Enabled on an Azure resource	Created as a standalone Azure resource
Lifecycle	Tied to the parent resource; deleted with it	Independent of the resources to which it is assigned
Association	One identity for one resource	Can be assigned to multiple supported resources
Suitable when	A workload has its own isolated identity	Identity lifecycle must be managed separately or shared deliberately
Governance note	Review permissions whenever the resource is recreated	Control reuse carefully to avoid excessive shared permissions

A system-assigned identity is usually straightforward for a single workload. A user-assigned identity is useful when identity lifecycle must be independent of compute lifecycle or when multiple resources intentionally share the same identity. Sharing should be deliberate and limited because all attached workloads inherit the identity's permissions.

5.4 Authentication and Authorization

Authentication and authorization are separate. Managed Identity enables the workload to prove its identity and obtain a token. The target service still requires an appropriate role assignment, database permission or service-specific authorization grant. The recommended design follows least privilege: grant the minimum required access at the narrowest practical scope.

5.5 Target Services

- Azure Key Vault: use a supported Microsoft Entra authentication path and assign an appropriate Key Vault data-plane role or access policy, depending on the vault's authorization configuration.
- Azure Storage: assign a narrowly scoped data role, such as a Blob data role, to the managed identity.
- Azure SQL Database: configure Microsoft Entra authentication and create or map the relevant database principal and permissions.

Support must be checked for the exact resource type, API, SDK and hosting environment. A service's presence in Azure does not automatically mean every protocol or legacy client library supports token-based authentication.

6. PROPOSED SYSTEM ARCHITECTURE

The proposed architecture replaces the application's use of a stored client secret with a platform-mediated token request. The application continues to call the target service, but authenticates with a short-lived access token obtained through its assigned managed identity.

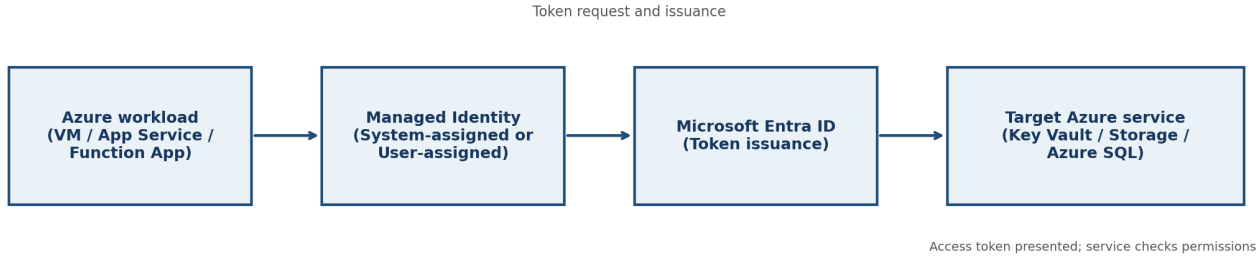


Figure 1. Conceptual managed-identity authentication flow.

6.1 Authentication Workflow

– An Azure workload (for example, an App Service, Function App or virtual machine) is configured with a system-assigned or user-assigned managed identity.

– The application requests an access token for the intended target resource using an Azure Identity SDK or the hosting platform's documented managed-identity endpoint. The application does not provide a client secret.

– The platform identity endpoint obtains the token through the managed identity infrastructure and Microsoft Entra ID.

– The application sends the access token to the target service over a protected connection.

– The target service validates the token and checks the identity's assigned permissions. Access is granted only when authentication and authorization both succeed.

6.2 Design Principles

• No embedded credentials: do not place client secrets, certificates or token values in source code or static configuration.

• Least privilege: assign only required roles and scopes; avoid broad subscription-level permissions unless justified.

• Identity separation: prefer distinct identities for unrelated workloads and environments.

• Explicit identity selection: where multiple user-assigned identities are attached, configure the application to select the intended identity explicitly.

• Resilience: use supported SDK credential chains, token caching and retry behaviour; avoid repeatedly calling the identity endpoint unnecessarily.

• Observability: retain platform activity, identity sign-in and target-service diagnostic logs where available.

7. MIGRATION METHODOLOGY

A controlled, phased migration is recommended. Each workload should be migrated and validated independently so that failures can be isolated and rollback remains practical.

7.1 Phase 1 — Discovery and Inventory

- Identify applications, hosting resources, owners, environments and downstream Azure dependencies.
- Record each service principal, credential type, credential expiry, consumers, permission grants and business criticality.
- Determine whether the workload runs on an Azure resource that supports managed identity.
- Confirm whether the target service, API and client library support Microsoft Entra token authentication.

7.2 Phase 2 — Identity Design

- Choose system-assigned identity for a workload-specific identity tied to one resource.
- Choose user-assigned identity when independent lifecycle or controlled reuse is required.
- Define identity naming, ownership, environment separation, tagging and permission-review standards.

7.3 Phase 3 — Provisioning and Authorization

- Enable or create the identity using the Azure portal, Azure CLI, PowerShell or infrastructure-as-code.
- Assign the minimum target-service permissions at the narrowest practical scope.
- Allow for propagation delay and verify role assignments before application cutover.

7.4 Phase 4 — Application Migration

- Replace client-secret-based credential construction with a supported managed-identity credential implementation.
- Remove secret references from application settings and deployment pipelines after confirming they are no longer needed by the workload.
- Use environment-specific configuration and explicit client/resource identifiers where required for user-assigned identities.

7.5 Phase 5 — Testing and Cutover

- Test successful token acquisition and access to each target service.
- Test denied access for an identity without the required role to confirm permissions are not overly broad.
- Test restart, scale-out, deployment-slot or failover behaviour as applicable to the hosting service.
- Monitor application and service logs through an agreed observation period.
- Revoke the old secret only after the new path is proven and the rollback window is closed.

7.6 Phase 6 — Decommissioning and Governance

- Remove obsolete credentials and unused permissions; update the inventory and ownership records.
- Establish periodic reviews for identity assignments, unused identities and excessive permissions.
- Document exceptions and define a remediation owner and review date for each one.

8. TOOLS AND TECHNOLOGIES

Component	Purpose in the proposed solution
Microsoft Entra ID	Identity directory and token issuer for supported authentication flows
Azure Managed Identity	Platform-managed workload identity; system-assigned or user-assigned
Azure Identity SDK	Application-side credential abstraction for supported languages and hosting environments
Azure Key Vault	Secure storage service accessed using Microsoft Entra authentication
Azure Storage	Object and file storage accessed using data-plane role assignments
Azure SQL Database	Relational database with Microsoft Entra authentication and database-level permissions
Azure RBAC	Role-based authorization for supported Azure resource operations
Azure Monitor / diagnostic logs	Operational monitoring and investigation, where configured
Azure CLI / PowerShell / IaC	Provisioning, role assignment and repeatable configuration

The exact SDK package, language, deployment pipeline and Azure resource types should be recorded during implementation. They were not specified in the source material and are therefore not assumed in this report.

9. SECURITY, RISK AND GOVERNANCE

9.1 Security Benefits

- Reduces the number of long-lived client secrets that applications must store and rotate.
- Reduces exposure through source repositories, environment configuration and deployment pipelines for migrated authentication paths.
- Provides an identity that can be assigned permissions and reviewed centrally.
- Supports a clearer relationship between an Azure workload and the identity used by that workload.

9.2 Residual Risks and Controls

Risk	Potential impact	Recommended control
Over-privileged identity	Unauthorized data or resource access	Least-privilege role assignments; periodic access review
Identity attached to compromised workload	Attacker may request tokens as that identity	Harden workload; restrict access; monitor anomalous activity
Incorrect identity selection	Access failures or unintended permissions	Explicitly select user-assigned identity when ambiguity exists
Role assignment delay or error	Temporary authorization failures	Validate assignments; include propagation time in deployment checks
Unsupported target or client	Migration cannot complete as designed	Compatibility assessment; retain approved exception path
Cross-tenant dependency	Managed identity cannot directly authenticate across directory boundary	Use an approved cross-tenant design or federation pattern; assess separately
Poor decommissioning	Old credentials remain usable	Track credential retirement and verify revocation

9.3 Governance Recommendations

- Maintain a central inventory linking each identity to its owner, workload, environment, target resources and permissions.
- Use separate identities for production and non-production workloads.
- Review role assignments and identity attachments at a defined cadence and after major application changes.
- Alert on unusual access patterns and unexpected authentication failures where diagnostic data is available.
- Protect administrative operations that create identities or grant permissions through role separation and change control.

10. BOTTLENECKS AND LIMITATIONS

10.1 Legacy and Third-Party Service Support

Some legacy services, third-party APIs or older client libraries accept only connection strings, API keys, passwords or certificates. If the target does not support Microsoft Entra authentication, managed identity cannot directly replace the credential for that integration. Where possible, update the service or client library. Otherwise, retain the credential in an approved secret-management system, restrict its access, rotate it and document the exception.

10.2 Cross-Tenant Scenarios

Microsoft documentation states that managed identities do not currently support direct cross-directory (cross-tenant) scenarios. A workload's managed identity should therefore not be assumed to authenticate directly to a resource in another Microsoft Entra tenant. Such requirements need a separately designed and approved mechanism, potentially involving a multitenant application, workload identity federation or another supported cross-tenant pattern. Guest access and federation are not automatic substitutes; their suitability depends on the target service and trust model.

10.3 Platform and Operational Constraints

- Only supported Azure resource types can be assigned managed identities; non-Azure hosting environments require another supported workload identity approach.
- Token acquisition endpoints, SDK behaviour and supported authentication flows vary by hosting platform and target service.
- Identity permissions and token caches may take time to reflect changes; operational procedures must account for propagation and caching behaviour.
- A managed identity does not protect an application from compromise. Code running with access to the workload's identity endpoint may be able to request tokens as that identity.

These limitations do not invalidate the approach. They define where managed identity is applicable and where an exception or alternative architecture is required.

11. VALIDATION AND TEST PLAN

The following test plan should be executed for each migrated workload. Actual results, dates, environment names and evidence links should be added to the project record after testing.

Test ID	Test case	Expected result / acceptance criterion
T-01	Identity is enabled and visible on the Azure resource	Correct identity is provisioned and associated with the intended workload
T-02	Application requests a token for the target resource	Token acquisition succeeds without a client secret in application configuration
T-03	Access Key Vault / Storage / SQL as applicable	Required operation succeeds using the assigned identity
T-04	Attempt an operation outside the granted role	Access is denied, confirming least-privilege boundaries
T-05	Restart or scale the workload	Authentication continues to work without manual secret injection
T-06	Inspect configuration and deployment variables	No obsolete client secret remains in the migrated authentication path
T-07	Review audit and diagnostic records	Relevant identity and target-service activity can be investigated
T-08	Revoke the old service-principal credential after cutover	Application continues operating through managed identity

11.1 Acceptance Criteria

- All in-scope compatible workloads authenticate to their target services using managed identity.
- No client secret or certificate is required by the migrated Azure-to-Azure authentication path.
- Permissions are limited to the documented business requirement.
- Positive and negative authorization tests pass.
- Monitoring and rollback procedures are documented.
- Every non-migrated workload has a recorded reason, owner and review action.

12. EXPECTED OUTCOMES AND BENEFITS

- Reduced secret-management effort for supported workloads by eliminating manual creation, distribution and rotation of the relevant client secrets.
- Reduced risk of long-lived credential leakage in application configuration and deployment processes.
- Improved identity governance through explicit workload identities and role assignments.
- More consistent authentication patterns across Azure-hosted applications.
- A documented exception register for legacy integrations, unsupported platforms and cross-tenant dependencies.

These are expected outcomes, not measured project results. Quantitative benefits should be reported only after baseline and post-migration measurements are collected.

13. PROJECT DELIVERABLES AND FUTURE ENHANCEMENTS

13.1 Planned Deliverables

– Application and credential inventory, including ownership and compatibility classification.

– Managed-identity design for each eligible workload.

– Updated application configuration and identity/role-assignment records.

– Test evidence covering token acquisition, service access, negative authorization and operational behaviour.

– Credential decommissioning record and exception register.

– Operational runbook covering provisioning, access review, troubleshooting and rollback.

13.2 Future Enhancements

• Automate identity provisioning and RBAC assignment using infrastructure-as-code.

• Introduce policy checks to detect new secret-based authentication where managed identity is supported.

• Create dashboards for identity ownership, role assignments, exceptions and authentication failures.

• Evaluate workload identity federation for suitable non-Azure or cross-cloud workloads.

• Measure credential count, rotation effort, migration coverage and authentication-related incidents before and after adoption.

14. CONCLUSION

Azure Managed Identity offers a practical way to remove long-lived application credentials from supported Azure-to-Azure authentication paths. By allowing the Azure platform to manage workload identity and token acquisition, applications can access compatible services without storing or rotating a client secret or certificate for that access.

Successful adoption nevertheless requires more than enabling an identity. Each workload must be inventoried, the target service and client library must support Microsoft Entra authentication, permissions must be assigned using least privilege, and the new authentication path must be tested before the old credential is revoked. Monitoring, ownership and periodic access reviews remain essential.

Legacy services, unsupported hosting environments and cross-tenant dependencies define the practical limits of the approach. These cases should be documented and addressed through an approved alternative rather than represented as completed managed-identity migrations. With phased execution and evidence-based validation, the proposed approach can reduce credential-management overhead and improve the security posture of the Azure application estate.

15. REFERENCES

• Microsoft. “Managed identities for Azure resources — Overview.” Microsoft Learn. https://learn.microsoft.com/entra/identity/managed-identities-azure-resources/overview

• Microsoft. “Managed identities for Azure resources — Frequently asked questions.” Microsoft Learn. https://learn.microsoft.com/entra/identity/managed-identities-azure-resources/managed-identities-faq

• Microsoft. “Developer introduction and guidelines for managed identities.” Microsoft Learn. https://learn.microsoft.com/en-us/entra/identity/managed-identities-azure-resources/overview-for-developers

• Microsoft. “How managed identities for Azure resources work with Azure virtual machines.” Microsoft Learn. https://learn.microsoft.com/en-us/entra/identity/managed-identities-azure-resources/how-managed-identities-work-vm

Note: Online documentation was consulted for technical terminology and platform limitations. The project-specific estate inventory, implementation evidence and measured results must be supplied by the project team.

APPENDIX A — IMPLEMENTATION EVIDENCE CHECKLIST

- [ ] Azure resource and environment inventory completed.
- [ ] Managed identity type and principal/object identifier recorded.
- [ ] Target service compatibility confirmed.
- [ ] RBAC role and scope documented.
- [ ] Application code/configuration change reviewed.
- [ ] Positive and negative test evidence attached.
- [ ] Logs and monitoring reviewed.
- [ ] Rollback plan tested or approved.
- [ ] Old service-principal secret/certificate revoked.
- [ ] Exception register and operational ownership updated.

APPENDIX B — TERMS AND ABBREVIATIONS

Term	Meaning
API	Application Programming Interface
Azure RBAC	Azure Role-Based Access Control
Entra ID	Microsoft Entra ID, formerly Azure Active Directory
OAuth 2.0	Authorization framework used to obtain and present access tokens
Managed Identity	Azure-managed workload identity for supported resources
Service Principal	Tenant-local identity representing an application
Token	Short-lived credential issued for a target resource
Workload	An application, service, job or process running on a compute resource